

GETAIBD

Developer API

Public API reference — call every model from your own apps

<https://ai-doer.com/v1/api>

OpenAI-compatible · billed in BDT credits · one key for every provider

Overview

The AI Doer Developer API lets you call any supported model from your own code, billed to your BDT credit balance. It is OpenAI-compatible, so the official OpenAI SDKs work by simply overriding the base URL.

Base URL

```
https://ai-doer.com/v1/api
```

Authentication

- Send your key as a Bearer token: Authorization: Bearer YOUR_API_KEY
- Keys start with “aiob_” and are created on the Integrations page of ai-doer.com.
- Every request is billed to your organization’s credit balance; revoke a key anytime.
- Python / Node: use the OpenAI SDK with base_url set to the URL above.

Quickstart

CHAT COMPLETION (CURL)

```
curl https://ai-doer.com/v1/api/chat/completions \
  -H "Authorization: Bearer YOUR_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "model": "gpt-4o",
    "messages": [{ "role": "user", "content": "Hello!" }],
    "stream": false
  }'
```

Endpoints

GET /models

List every model your key can call, with its id, modality, capabilities and context window. Use a returned id as the “model” field in the other endpoints.

EXAMPLE RESPONSE

```
{
  "data": [
    {
      "id": "gpt-4o",
      "name": "GPT-4o",
      "modality": "chat",
      "capabilities": ["streaming", "tools", "image-out"],
      "context_window": 128000,
      "max_output_tokens": 16384,
      "credits_per_output_token": 12.0,
      "free": false,
      "byok_required": false
    }
  ]
}
```

GET /balance

Check the remaining credit balance for the organization that owns the key.

EXAMPLE RESPONSE

```
{
  "credits_balance": 1240.75
}
```

POST /chat/completions

OpenAI-compatible chat completions. Works with the official OpenAI SDKs by pointing the base URL at AI Doer. Streams by default; set "stream": false for a single JSON response. Supports tools / function calling.

EXAMPLE REQUEST BODY

```
{
  "model": "gpt-4o",
  "messages": [
    { "role": "system", "content": "You are a helpful assistant." },
    { "role": "user", "content": "Write a haiku about Dhaka." }
  ],
  "stream": false
}
```

EXAMPLE RESPONSE

```
{
  "id": "chatcmpl-9f3...",
  "object": "chat.completion",
  "model": "gpt-4o",
  "choices": [
    {
      "index": 0,
      "message": { "role": "assistant", "content": "Rickshaw bells ringing..." },
      "finish_reason": "stop"
    }
  ],
  "usage": { "prompt_tokens": 24, "completion_tokens": 18, "total_tokens": 42 }
}
```

- When streaming (the default), the response is server-sent events: lines of "data: { ...chunk...}" ending with "data: [DONE]".
- Each chunk follows the OpenAI chat.completion.chunk shape with choices[].delta.

POST /generate

Simple one-shot text generation. Send a prompt (or a messages array) and get the text back plus the credits spent and your new balance — handy when you don't need the full chat schema.

EXAMPLE REQUEST BODY

```
{
  "model": "gpt-4o",
  "prompt": "Summarize the benefits of solar power in two sentences."
}
```

EXAMPLE RESPONSE

```
{
  "model": "gpt-4o",
  "text": "Solar power is clean, renewable and increasingly cheap...",
  "usage": { "input_tokens": 14, "output_tokens": 38 },
  "credits_spent": 6.2,
  "credits_balance": 1234.55
}
```

POST /embeddings

OpenAI-compatible embeddings for search and retrieval (RAG). Pass a single string or an array of strings; returns one vector per input.

EXAMPLE REQUEST BODY

```
{
  "input": ["AI Doer makes AI affordable in Taka.", "Another sentence."],
  "model": "text-embedding-3-small"
}
```

EXAMPLE RESPONSE

```
{
  "object": "list",
  "model": "text-embedding-3-small",
  "data": [
    { "object": "embedding", "index": 0, "embedding": [0.0123, -0.0456, "..."] },
    { "object": "embedding", "index": 1, "embedding": [0.0210, 0.0099, "..."] }
  ],
  "usage": { "prompt_tokens": 12, "total_tokens": 12 }
}
```